🧪 Lab: Deploy a Model Using Triton via NGC Notebook (Simulated Environment)

> 💡 **Goal:** Learn how to deploy and serve a trained model using **NVIDIA Triton Inference Server** via an **NGC-hosted Jupyter Notebook** (or a local simulation using Google Colab + Docker).

🧰 Prerequisites

- NGC (NVIDIA GPU Cloud) account → https://ngc.nvidia.com
- Basic familiarity with Jupyter Notebooks
- (Optional) Docker installed if running locally
- (Optional) GPU instance or Google Colab Pro for acceleration

---

✅ STEP 1: Access the Official NGC Triton Sample Notebook

1. Go to https://catalog.ngc.nvidia.com
2. In the **Search bar**, type: `Triton Inference Server`
3. Filter by **Jupyter Notebooks** or click the "Collections" tab.
4. Look for **Triton Inference Server Sample Notebooks**.
5. Select the notebook titled:
   📓 `Quick Start - Deploying a Model with Triton`

📎 **Explanation:**
NGC hosts pre-configured Jupyter notebooks that run on cloud GPUs. These environments include CUDA, TensorRT, and Triton pre-installed, saving you setup time.

✅ STEP 2: Launch the Notebook in the NGC Environment

1. Click **"Launch"** on the notebook.
2. Select a runtime (preferably one with GPU).
3. Once the Jupyter environment loads, open the `Triton Quick Start.ipynb`.

📎 **Explanation:**
This notebook demonstrates how to pull a model, serve it using Triton, and send inference requests—all in one place.

✅ STEP 3: Understand the Directory Structure for Triton

In the notebook or terminal, observe the following structure:

```
model_repository/
 └── resnet50/
     └── 1/
         └── model.onnx
     └── config.pbtxt
```

📎 **Explanation:**

Triton expects a **Model Repository**, where each model:

- Has its own directory (e.g., `resnet50`)
- Has **versioned subfolders** (`1/`, `2/`, etc.)
- Contains a **config.pbtxt** file for metadata

✅ STEP 4: Pull and Prepare the Model

The notebook typically includes:

```
!mkdir -p model_repository/resnet50/1
!wget https://.../resnet50.onnx -O model_repository/resnet50/1/model.onnx
```

You may also define a minimal `config.pbtxt`:

```
name: "resnet50"
platform: "onnxruntime_onnx"
max_batch_size: 8
input [
  {
    name: "input_tensor"
    data_type: TYPE_FP32
    dims: [3, 224, 224]
  }
]
output [
  {
    name: "output_tensor"
    data_type: TYPE_FP32
    dims: [1000]
  }
]
```

📎 **Explanation:**

This configuration tells Triton how to handle inference: expected input/output shapes, data types, and batching behavior.

✅ STEP 5: Launch the Triton Inference Server

Use a cell in the notebook to run:

```
!docker run --rm --gpus=all \
  -p8000:8000 -p8001:8001 -p8002:8002 \
```

```
-v $PWD/model_repository:/models \
nvcr.io/nvidia/tritonserver:24.03-py3 \
tritonserver --model-repository=/models
```

📎 **Explanation:**
This command:

- Pulls the **Triton Docker image** from NGC
- Mounts your model repository
- Exposes Triton ports:
  - `8000` : HTTP
  - `8001` : gRPC
  - `8002` : Metrics

✅ STEP 6: Send an Inference Request to Triton

In the notebook:

```python
import tritonclient.http as httpclient
import numpy as np

client = httpclient.InferenceServerClient(url="localhost:8000")

inputs = httpclient.InferInput("input_tensor", [1, 3, 224, 224], "FP32")
inputs.set_data_from_numpy(np.random.rand(1, 3, 224, 224).astype(np.float32))

outputs = httpclient.InferRequestedOutput("output_tensor")

response = client.infer(model_name="resnet50", inputs=[inputs], outputs=[outputs])
print(response.get_response())
```

📎 **Explanation:**
This Python code uses Triton's client SDK to:

- Generate random input data
- Send it to the model via REST API
- Print the model's output predictions

✅ STEP 7: Validate and Monitor

You can monitor performance by visiting:

- `http://localhost:8002/metrics` → for Prometheus-formatted metrics
- Use `nvidia-smi` or `dcgmi` to see GPU utilization

📎 **Explanation:**
Triton exposes real-time stats on inference throughput, memory usage, latency, and errors.

🧪 Optional Challenge

Try switching from ONNX to a PyTorch model, or increase batch size to see performance differences using dynamic batching in `config.pbtxt`.

🏁 Conclusion

By completing this lab, you've:

- Explored Triton's model repository structure
- Served a model using Triton in Docker
- Sent inference requests using the Python SDK
- Monitored GPU and inference performance metrics

📌 This workflow mimics real-world AI deployment patterns used in **data centers**, **edge systems**, and **Kubernetes clusters**—and is highly relevant for the **NCA-AIIO exam**.
